

智能巡检车 PRO 版实验手册

机器人仿真设计及应用

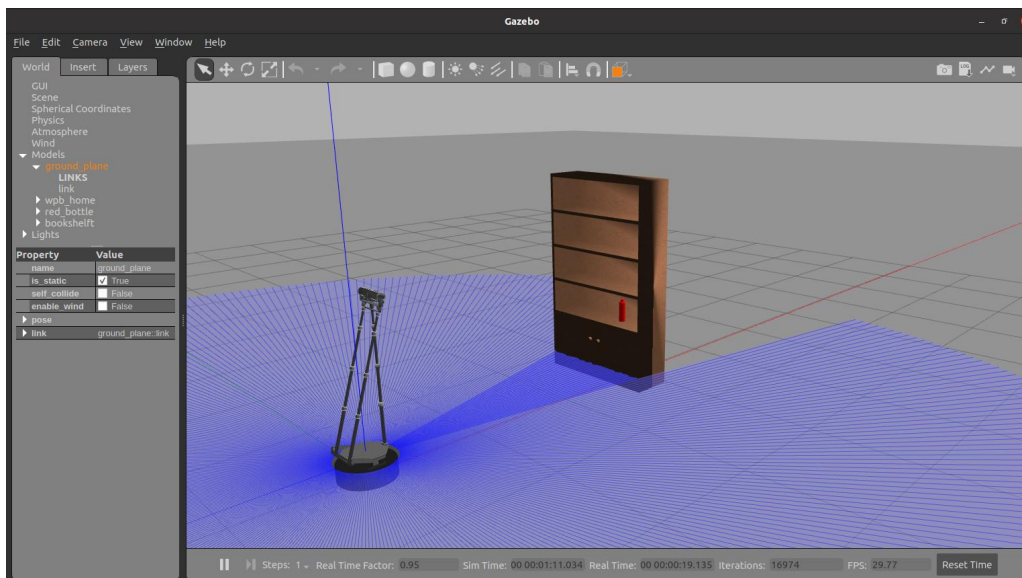
目录

1 机器人仿真概念	2
1.1 什么是机器人系统仿真	2
2 URDF 集成 Rviz	5
2、最终结果图示	6
3、具体操作步骤	6
3 URDF 语法	9
3.1 URDF 的基本概念	9
3.2 robot 标签	10
6、URDF 工具	15
4 URDF 优化	16
4.1 URDF 文件构建机器人模型的过程中的问题	16
5 Rviz 中控制机器人模型运动	19
5.1 Arbotix 介绍	19
6 URDF 集成 Gazebo	21
6.1 URDF 与 Gazebo 集成流程步骤	21
6.2 URDF 集成 Gazebo 的相关设置	23
6.3 Gazebo 仿真环境搭建	24
7 URDF、Gazebo 和 Rviz 的综合应用	27
7.2 雷达信息仿真以及显示	29
7.3 摄像头信息仿真以及显示	30
7.4 深度相机信息仿真以及显示	32

1 机器人仿真概念

1.1 什么是机器人系统仿真

机器人系统仿真通过计算机对实体机器人系统进行模拟的技术。在 ROS 中，仿真实现涉及的内容主要有以下三个系统性实现：（1）使用 URDF 对机器人建模、（2）使用 Gazebo 创建仿真环境、（3）使用 Rviz 感知环境。



1.2 机器人系统仿真的优势

仿真在机器人系统研发过程中占有举足轻重的地位，在研发与测试中较之于实体机器人实现，

仿真有如下几点的显著优势：

（1）、低成本。仿真可以减少对机器人实物的依赖，大大降低研发成本。

（2）、高效率。仿真环境灵活多样，不受实物限制，运用得当，可以有效提高测试的效率及覆盖率。

（3）、高安全性。未经验证的程序运行到机器人实体上可能有安全问题，甚至可能损坏机器人，带来经济或者安全损失；而在仿真环境下就没有这些问题，可以放心测试，测试充分

再部署到真实机器人，更安全可靠。

1.3 机器人系统仿真存在的不足

机器人在仿真环境与实际环境下的表现差异较大，仿真环境一般比较理想，并不能完全做到模拟真实的物理世界，存在一些"失真"的情况，原因：

- (1)、仿真器所使用的物理引擎目前还不能够完全精确模拟真实世界的物理情况。
- (2)、仿真器构建的是关节驱动器（电机和齿轮箱）、传感器与信号通信的绝对理想情况，目前不支持模拟实际硬件有缺陷或者一些临界状态等情形。

1.4 机器人仿真的组件构成

机器人仿真通过使用 URDF、Gazebo、Rviz 三者配合进行系统性的实现。其中 URDF 组件用于创建机器人模型，Gazebo 组件用于搭建仿真环境，Rviz 组件用于图形化地显示机器人各种传感器感知到的环境信息。下面分别介绍：

(1)、URDF

URDF 全称（Unified Robot Description Format），即统一（标准化）机器人描述格式，是一个 XML 语法框架下用来描述机器人的语言格式，以一种 XML 的方式描述机器人的部分结构，比如底盘、摄像头、激光雷达、机械臂以及不同关节的自由度等，对机器人建模，然后放到 ROS 里面进行仿真与分析。

(2)、Gazebo：

Gazebo 是一款开源的 3D 动态模拟器，用于显示机器人模型并创建仿真环境，能够在复杂的室内和室外环境中准确有效地模拟机器人。与游戏引擎提供高保真度的视觉模拟类似，Gazebo 提供高保真度的物理模拟，其提供一整套传感器模型，以及对用户和程序非常友好的交互方式。Gazebo 为研究人员和工程师提供了一个平台，以在无需实际硬件的情况下开发和测试机器人及相关算法。它被广泛用于以下领域和应用：

- 机器人和算法设计测试：Gazebo 提供了一个实际的环境，使得开发人员能快速地测试和验证新的机器人设计和算法。它可以精确并高效地模拟在复杂的室内和室外环境中的机器人行为。
- 数据可视化和遥感环境模拟：Gazebo 可以用于数据可视化，远程环境模拟，甚至是黑盒系统的逆向工程。
- 物理设计迭代：Gazebo 允许用户在真实环境中快速迭代新的物理设计，并提供高保真度的传感器流数据，以测试控制策略并安全地利用模拟进行持续集成测试。
- 无人机和人形机器人模拟：Gazebo 能模拟无人机和人形机器人等多种机器人类型的行为和性能。
- 技术挑战和竞赛：Gazebo 被用作多种技术挑战和竞赛的模拟环境，例如 DARPA Robotics Challenge 和 NASA Space Robotics Challenge。在这些竞赛中，Gazebo 用于模拟危险或人工设计的环境，以测试半自主地面机器人的性能和能力。

(3)、Rviz：

Rviz (ROS Visualization Tool)，是一个用于可视化机器人系统的开源工具，用于显示和调试机器人的传感器数据、状态信息和运动规划等。它是 ROS (Robot Operating System) 的一部分，是 ROS 中最常用的可视化工具之一。它以三维方式显示 ROS 消息，对数据进行可视化表达。例如：可以显示机器人模型，可以表达激光测距传感器的障碍物距离数据、Kinect 等三维距离传感器的点云数据 (PCD, Point Cloud Data)、从相机获取的图像数据等。Rviz 的主要特点如下：

- 可视化机器人模型：Rviz 可以加载和显示机器人的三维模型，并根

据实际的关节状态进行动态更新。这使得用户能够直观地了解机器人的外观和姿态。

- 显示传感器数据：Rviz 可以接收和显示来自机器人传感器（如激光雷达、摄像头、IMU 等）的数据。用户可以实时查看和分析传感器数据，帮助理解机器人周围的环境。
- 生成导航地图：Rviz 可以通过接收来自 SLAM（ Simultaneous Localization and Mapping ）或其他建图算法的数据，生成并显示机器人所在环境的二维或三维地图。
- 调试运动规划：Rviz 可以显示机器人的路径规划结果，并提供交互式界面来调试和优化运动规划算法。用户可以可视化虚拟路径、障碍物和碰撞检测等信息。
- 可定制性：Rviz 提供了丰富的配置选项，允许用户按照自己的需求自定义界面布局、可视化对象和颜色风格等。用户可以根据实际情况进行个性化设置，以满足特定的可视化需求。

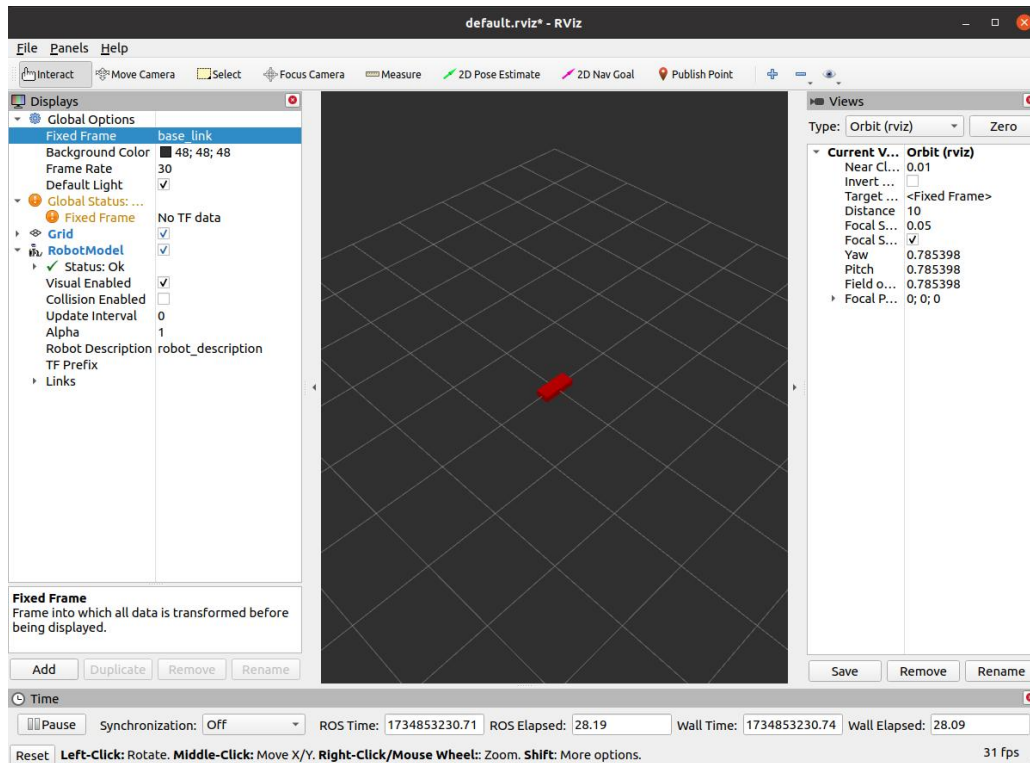
2 URDF 集成 Rviz

URDF 只是一个符合固定格式的文件，需要结合 Rviz 或 Gazebo，在 Rviz 或 Gazebo 中渲染成图形化的机器人模型。下面以最简单的 URDF 与 Rviz 的集成使用例子来让大家对这种配合方式有个了解。

1、需求描述

使用 URDF 文件描述一个长方体形状的机器人，并在 Rviz 中显示出来。

2、最终结果图示



3、具体操作步骤

步骤 1、创建功能包，导入依赖。

使用 ROS 指令创建一个新的功能包，名称自定义，导入依赖包：urdf 与 xacro

```
catkin_create_pkg urdf_test urdf xacro
```

在新建功能包目录 urdf_test 下，再新建如下几个子目录：

urdf、meshes、config、launch。

步骤 2、编写 URDF 文件。

在 urdf 文件夹中添加一个名为 mycar.urdf 的文件，写入类似如下内容：

```
<robot name="mycar">
  <link name="base_link">
    <visual>
```

```
<geometry>
  <box size="0.5 0.2 0.1" />
</geometry>
</visual>
</link>
</robot>
```

步骤 3、在 ROS 的 launch 文件中集成 URDF 与 Rviz。

在 launch 目录下，新建一个 launch 文件 simulation.launch，该文件需要启动 Rviz，并导入 urdf 文件。

Rviz 启动后可以自动载入解析 urdf 文件，并显示机器人模型。

在 ROS 中，可以将 urdf 文件的路径设置到参数服务器，使用的参数名是:robot_description，示例代码如下：

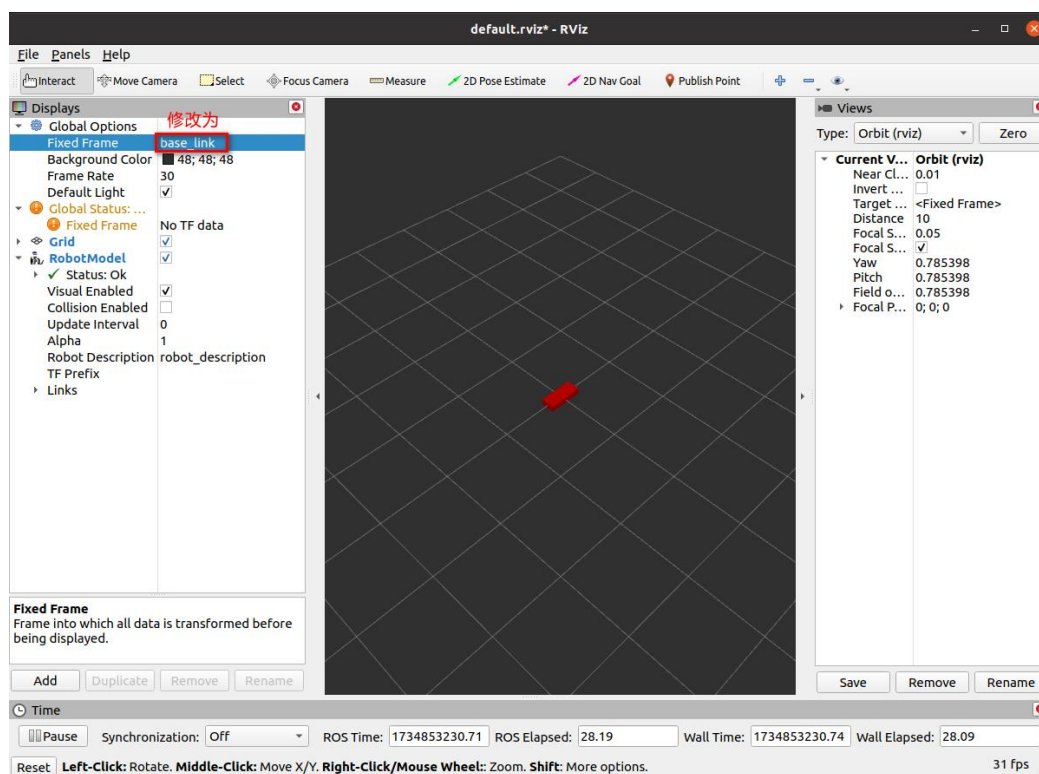
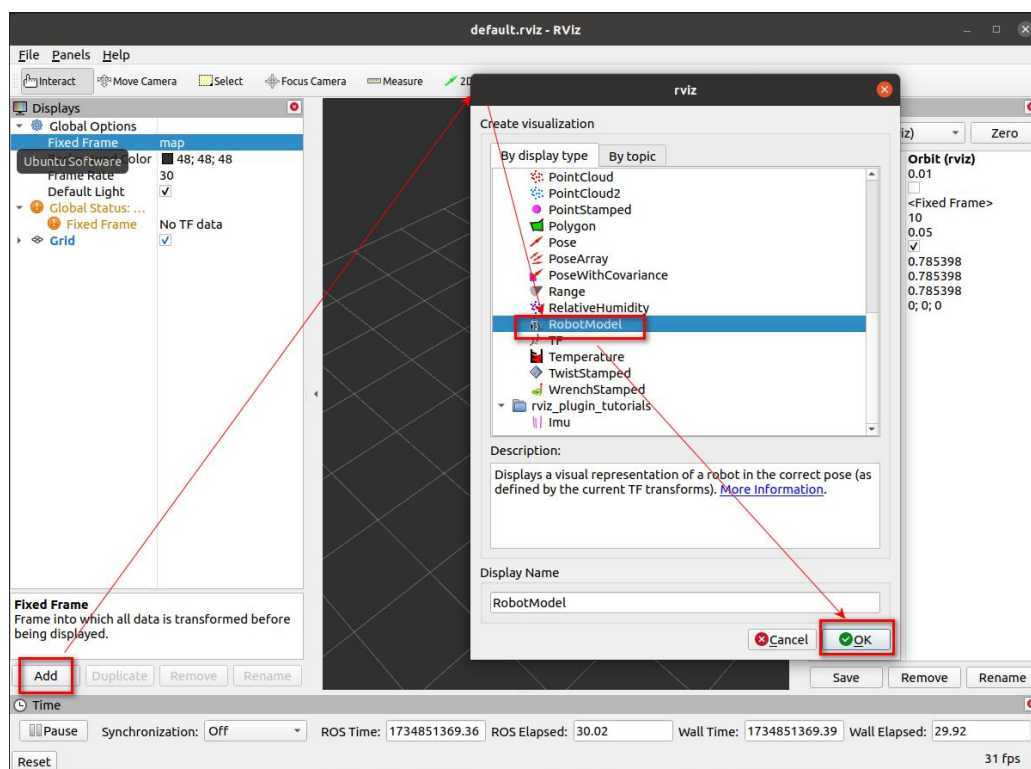
```
<launch>
  <param name="robot_description" textfile="$(find mycar)/urdf/mycar.urdf" />
  <node pkg="rviz" type="rviz" name="rviz" />
</launch>
```

步骤 4、在 Rviz 中显示机器人模型

运行 ROS 指令启动 Rviz：

```
roslaunch urdf_test simulation.launch
```

Rviz 启动后，会发现并没有出现机器人模型，这是因为默认情况下没有添加机器人显示组件，需要手动添加，添加方式如下图：

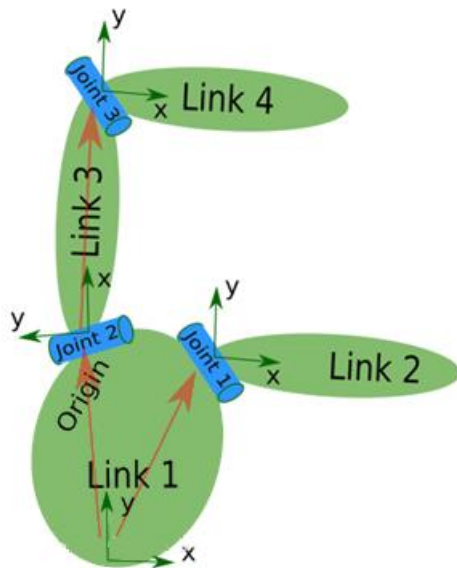


按图示设置完毕后，可以在 Rviz 中看到长方体形状的机器人了。

3

URDF 语法

3.1 URDF 的基本概念



从机构学角度讲，机器人通常被分解为由连杆(link)和关节(joint)组成的结构。连杆是带有质量属性的刚体，而关节是连接和限制两个刚体相对运动的结构，也被称为运动副。通过关节将连杆一次连接起来，就构成了一个个运动链，也就是机器人的机构模型。URDF 就是描述这一系列关节和连杆的相对关系的工具。除此之外，还包括惯性属性、几何特点和碰撞模型等一系列附加参数。

URDF 文件中有<root>根节点、<joint>和<link>子节点。<joint>对应关节。<link>对应连杆。组合在一起，形成机器人的完整模型。

在基础模型之上，我们为机器人添加尺寸大小。

由于每个环节的参考系都位于该环节的底部，关节也是如此，所以在表示尺寸大小时，只需要描述其相对于连接的关节的相对位置关系即可。

URDF 中的 origin 标签就是用来表示这种相对关系。

```
<?xml version="1.0" ?>
<robot name="jubot">

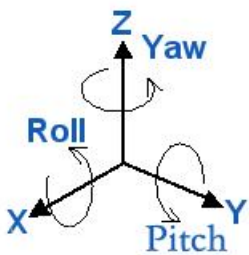
<!--base_car-->
  <link name="base_link">
    <visual>
      <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
      <geometry>
        <cylinder radius="0.20" length="0.16"/>
      </geometry>
      <material name="yellow">
        <color rgba="1 0.4 0 1"/>
      </material>
    </visual>
  </link>

  <!--left_wheel-->
    <joint name="left_wheel_joint" type="continuous">
      <origin xyz="0.0 0.19 -0.05" rpy="0.0 0.0 0.0"/>
      <parent link="base_link"/>
      <child link="left_wheel_link"/>
      <axis xyz="0.0 1.0 0.0"/>
    </joint>

    <link name="left_wheel_link">
      <visual>
        <origin xyz="0.0 0.0 0.0" rpy="1.5707 0.0 0.0"/>
        <geometry>
          <cylinder radius="0.06" length="0.025"/>
        </geometry>
        <material name="white">
          <color rgba="1 1 1 0.9"/>
        </material>
      </visual>
    </link>
```

如果我们为机器人的关节添加 axis 旋转轴参数，那么该机器人模型就可以具备基本的运动学参数。

关于参数欧拉角 rpy，是 roll（滚转角）、pitch（俯仰角）、yaw（偏航角），分别对应绕 x 轴、y 轴、z 轴的角度。



3.2 robot 标签

urdf 中为了保证 xml 语法的完整性, 使用了 robot 标签作为根标签, 所有的 link 和 joint 以及其他标签都必须包含在 robot 标签内。在该标签内可以通过 name 属性设置机器人模型的名称。

(1) 属性

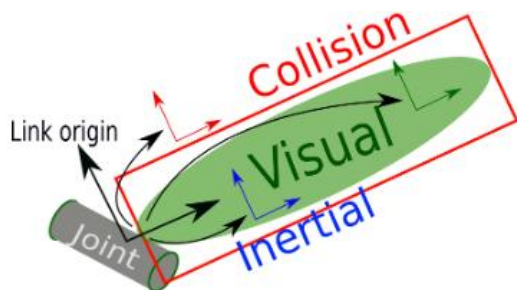
name: 指定机器人模型的名称。

(2) 子标签

其他标签都是 robot 标签的子级标签。

3.3 link 标签

urdf 中的 link 标签用于描述机器人某个部件(也即刚体部分)的外观和物理属性, 比如: 机器人底座、轮子、激光雷达、摄像头...每一个部件都对应一个 link, 在 link 标签内, 可以设计该部件的形状、尺寸、颜色、惯性矩阵、碰撞参数等一系列属性。



(1) 属性

name: 连杆名称

(2) 子标签

visual: 描述外观(对应的数据是可视的)

子标签 geometry: 设置连杆形状

子标签 origin: 设置偏移量与倾斜弧度

子标签 material: 设置材料属性 (颜色)

collision: 连杆的碰撞属性

inertial: 连杆的惯性矩阵

```
<!--base_car-->
  <link name="base_link">
    <visual>
      <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
      <geometry>
        <cylinder radius="0.20" length="0.16"/>
      </geometry>
      <material name="yellow">
        <color rgba="1 0.4 0 1"/>
      </material>
    </visual>
  </link>
```

(3) visual 标签的子标签

geometry 子标签: 设置连杆的形状

标签 1: box(盒状) 属性:size=长(x) 宽(y) 高(z)

标签 2: cylinder(圆柱) 属性:radius=半径 length=高度

标签 3: sphere(球体) 属性:radius=半径

标签 4: mesh(为连杆添加皮肤) 属性: filename=资源路径(格式:package://<packagename>/<path>/文件)

origin 属性: 设置偏移量和倾斜弧度

属性 1: xyz=x 偏移 y 偏移 z 偏移

属性 2: rpy=x 翻滚 y 俯仰 z 偏航 (单位是弧度)

material 属性及子标签

属性: name

标签: color

属性: rgba=红绿蓝权重值与透明度 (每个权重值以及透明度取值范围[0,1])

3.4 joint 标签

机器人的两个部件(分别称之为 parent link 与 child link)以“关节”(joint)的形式相连接。

joint 标签用于描述机器人关节的运动学和动力学属性, 还可以指定关节运动的安全极限。

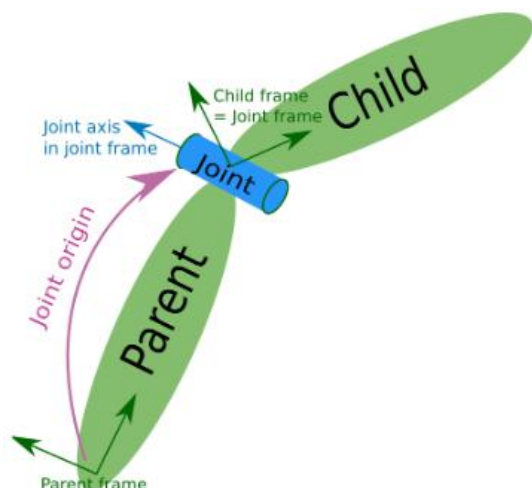
不同的关节有不同的运动形式: 旋转、滑动、固定、旋转速度、旋转角度限制....

```
<robot name="jubot">

  <!--base_car-->
    <link name="base_link">
      <visual>
        <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0"/>
        <geometry>
          <cylinder radius="0.20" length="0.16"/>
        </geometry>
        <material name="yellow">
          <color rgba="1 0.4 0 1"/>
        </material>
      </visual>
    </link>

    <!--left_wheel-->
      <joint name="left_wheel_joint" type="continuous">
        <origin xyz="0.0 0.19 -0.05" rpy="0.0 0.0 0.0"/>
        <parent link="base_link"/>
        <child link="left_wheel_link"/>
        <axis xyz="0.0 1.0 0.0"/>
      </joint>

      <link name="left_wheel_link">
        <visual>
          <origin xyz="0.0 0.0 0.0" rpy="1.5707 0.0 0.0"/>
          <geometry>
            <cylinder radius="0.06" length="0.025"/>
          </geometry>
          <material name="white">
            <color rgba="1 1 1 0.9"/>
          </material>
        </visual>
      </link>
```



(1) 属性

name: 关节名称

type: 关节运动形式，有以下取值：

continuous: 旋转关节，可以绕单轴无限旋转。

revolute: 旋转关节，类似于 continuous, 但是有旋转角度限制。

prismatic: 滑动关节，沿某一轴线移动的关节，有位置极限。

planer: 平面关节，允许在平面正交方向上平移或旋转。

floating: 浮动关节，允许进行平移、旋转运动。

fixed: 固定关节，不允许运动的特殊关节。

(2) 子标签

parent(必需的):

link 属性: 父级连杆的名字，是这个父级 link 在机器人结构树中的名字。

child(必需的):

link 属性: 子级连杆的名字，是这个子级 link 在机器人结构树中的名字。

origin:

xyz 属性: 各轴线上的偏移量。

rpy 属性: 各轴线上的偏移弧度。

axis

xyz 属性: 用于设置围绕哪个关节轴运动。

3.5 代码样例:

```
<robot name="mycar">
  <!-- 底盘 -->
  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.5 0.2 0.1" />
      </geometry>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <material name="blue">
        <color rgba="0 0 1.0 0.5" />
      </material>
    </visual>
  </link>
  <!-- 摄像头 -->
  <link name="camera">
    <visual>
      <geometry>
        <box size="0.02 0.05 0.05" />
      </geometry>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <material name="red">
        <color rgba="1 0 0 0.5" />
      </material>
    </visual>
  </link>
  <!-- 关节 -->
  <joint name="camera2baselink" type="continuous">
    <parent link="base_link"/>
    <child link="camera" />
    <!-- 需要计算两个 link 的物理中心之间的偏移量 -->
    <origin xyz="0.2 0 0.075" rpy="0 0 0" />
    <axis xyz="0 0 1" />
  </joint>
</robot>
```

6、URDF 工具

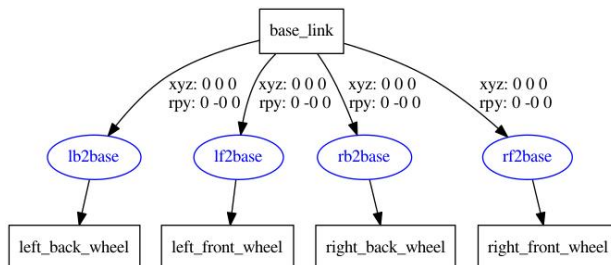
(1) check_urdf 语法检查工具

进入 urdf 文件所属目录，调用 check_urdf urdf 文件，如果不抛出异常，说明文件语法格式合法，否则非法。

```
ros@ubuntu:~/catkin_ws/src/urdf_test/urdf$ check_urdf mycar.urdf
robot name is: mycar
----- Successfully Parsed XML -----
root Link: base_link has 0 child(ren)
```

(2) urdf_to_graphviz 结构查看工具

进入 urdf 文件所属目录，调用 urdf_to_graphviz urdf 文件，当前目录下会生成 pdf 文件。



4 URDF 优化

4.1 URDF 文件构建机器人模型的过程中的问题

存在以下问题：

(1) 在设计关节的位置时，需要按照一定的公式计算，公式是固定的，但是在 URDF 中依赖于人工计算及录入，存在不便，容易计算失误，且当某些参数发生改变时，还需要重新计算。

(2) URDF 中的部分内容是高度重复的，驱动轮与支撑轮的设计实现，不同轮子只是部分参数不同，形状、颜色、翻转量都是一致的，在实际应用中，构建复杂的机器人模型时，更

是易于出现高度重复的设计，按照一般的编程理念，涉及到重复代码应该考虑封装以提升效率及安全性。

4.2 Xacro 简介

在 ROS 中给出了类似可编程的优化方案，称为：Xacro，通过变量结合函数直接解决上述问题，

(1) Xacro 概念

Xacro 是 XML Macros 的缩写，Xacro 是一种 XML 宏语言，是可编程的 XML。

(2) Xacro 原理

Xacro 可以声明变量，可以通过数学运算求解，使用流程控制来控制执行顺序，还可以通过类似函数的实现，封装固定的逻辑，将逻辑中需要的可变的数据以参数的方式暴露出去，从而提高代码复用率以及程序的安全性。

(3) Xacro 的作用

较之于纯粹的 URDF 实现，可以编写更安全、精简、易读性更强的机器人模型文件，且可以提高编写效率。

4.3 Xacro 语法

Xacro 提供了可编程接口，类似于编程语言，包括变量声明与调用、函数声明与调用等语法实现。

在使用 xacro 生成 urdf 时，根标签 robot 中必须包含命名空间声明：

xmlns:xacro=<http://wiki.ros.org/xacro>

类似如下：

```
<robot name="xxx" xmlns:xacro="http://wiki.ros.org/xacro">
    .....
</robot>
```

1、属性与算数运算

用于封装 URDF 中的一些字段，比如：PI 值，小车的尺寸，轮子半径

(1) 属性定义

```
<xacro:property name="xxxx" value="yyyy" />
```

(2) 属性调用

$\{属性名称\}$ ，比如：

```
<cylinder radius="${wheel_radius}" length="${wheel_length}" />
```

(3) 算数运算

$\{数学表达式\}$ ，比如：

```
<origin xyz="0 0 0" rpy="${PI / 2} 0 0" />
```

2、宏

类似于函数，提高代码复用率，优化代码结构，提高安全性

(1) 宏定义

```
<xacro:macro name="宏名称" params="参数列表(多参数之间使用空格分隔)">
.....
    参数调用格式: ${参数名}
</xacro:macro>
```

(2) 宏调用

```
<xacro:宏名称 参数 1=xxx 参数 2=xxx/>
```

3、文件包含

机器人由多部件组成，不同部件可能封装为单独的 xacro 文件，最后再将不同的文件集成，组合为完整机器人，可以使用文件包含实现。

4.4 Xarco 完整使用流程

(1)、编写 Xacro 文件

(2)、集成 launch 文件

方式 1: 先将 xacro 文件转换成 urdf 文件, 然后集成。

使用 ROS 指令先将 xacro 文件解析成 urdf 文件:

```
roslaunch xacro xacro xxx.xacro > xxx.urdf
```

然后再按照之前的集成方式直接整合。

方式 2: 在 launch 文件中直接加载 xacro(建议使用)。

加载 robot_description 时使用 command 属性, 属性值就是调用 xacro 功能包的 xacro 程序直接解析 xacro 文件。

launch 内容示例:

```
<param name="robot_description" command="$(find xacro)/xacro  
$(find urdf_test)/urdf/xacro/mycar.urdf.xacro" />
```

5

Rviz 中控制机器人模型运动

5.1 Arbotix 介绍

Arbotix 是一款控制电机、舵机的控制板, 并提供相应的 ros 功能包, 这个功能包的功能不仅可以驱动真实的 Arbotix 控制板, 它还提供一个差速控制器, 通过接受速度控制指令更新机器人的 joint 状态, 从而帮助我们实现机器人在 Rviz 中的运动。这个差速控制器在 arbotix_python 程序包中, 完整的 arboti 程序包还包括多种控制器, 分别对应 dynamixel 电机、多关节机械臂以及不同形状的夹持器。

5.2 实现流程

(1) 安装 Arbotix。

sudo apt-get install ros-noetic-arbotix

(2) 创建新功能包，准备机器人 urdf、xacro 文件。

调用 urdf 和 xacro 完成机器人的模型构造。

(3) 添加 Arbotix 配置文件。

添加 arbotix 所需的控制器配置文件，代码样例如下：

```
controllers: {  
  # 单控制器设置  
  base_controller: {  
    #类型: 差速控制器  
    type: diff_controller,  
    #参考坐标  
    base_frame_id: base_footprint,  
    #两个轮子之间的间距  
    base_width: 0.2,  
    #控制频率  
    ticks_meter: 2000,  
    #PID 控制参数，使机器人车轮快速达到预期速度  
    Kp: 12,  
    Kd: 12,  
    Ki: 0,  
    Ko: 50,  
    #加速限制  
    accel_limit: 1.0  
  }  
}
```

(4) 编写 launch 文件配置 Arbotix

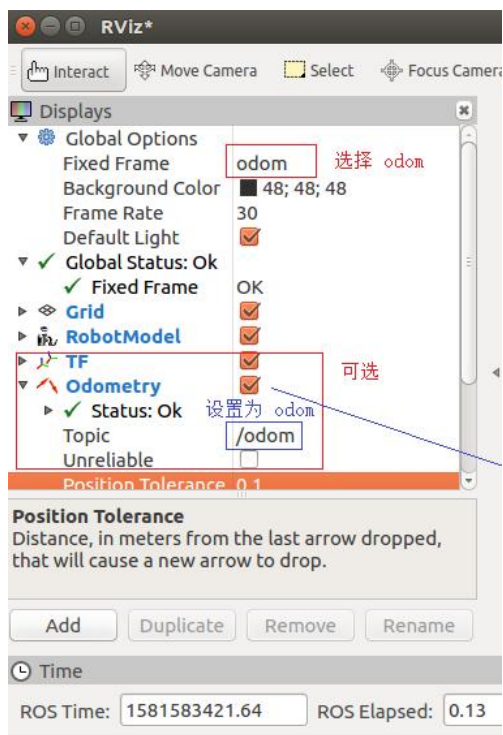
```
<node name="arbotix" pkg="arbotix_python" type="arbotix_driver"  
output="screen">  
  <rosparam file="$(find urdf_test)/config/hello.yaml" command="load" />  
  <param name="sim" value="true" />  
</node>
```

(5) 启动 launch 文件并控制机器人模型运动

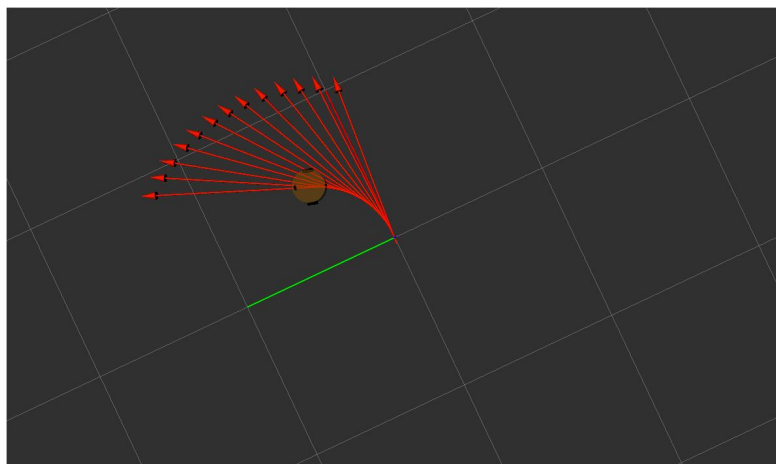
ROS 中启动 launch:

```
roslaunch xxxx ....launch
```

参见下图配置 Rviz:



可看到小车的运动，如下图：



6 URDF 集成 Gazebo

6.1 URDF 与 Gazebo 集成流程步骤

URDF 与 Gazebo 集成流程与 Rviz 实现类似，主要步骤如下：

步骤 1、创建功能包。

需要导入依赖包：urdf、xacro、gazebo_ros、gazebo_ros_control、gazebo_plugins

ROS 指令类似如下：

```
catkin_create_pkg urdf_gazebo urdf xacro gazebo_ros gazebo_ros_control  
gazebo_plugins
```

步骤 2、编写 URDF 或 Xarco 文件，类似下图。

```
<robot name="mycar">  
  <link name="base_link">  
    <visual>  
      <geometry>  
        <box size="0.5 0.2 0.1" />  
      </geometry>  
      <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0" />  
      <material name="yellow">  
        <color rgba="0.5 0.3 0.0 1" />  
      </material>  
    </visual>  
    <collision>  
      <geometry>  
        <box size="0.5 0.2 0.1" />  
      </geometry>  
      <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0" />  
    </collision>  
    <inertial>  
      <origin xyz="0 0 0" />  
      <mass value="6" />  
      <inertia ixx="1" ixy="0" ixz="0" iyy="1" iyz="0" izz="1" />  
    </inertial>  
  </link>  
  <gazebo reference="base_link">  
    <material>Gazebo/Black</material>  
  </gazebo>  
</robot>
```

步骤 3、启动 Gazebo 并显示机器人模型

launch 文件实现类似如下：

```
<launch>  
  <!-- 将 Urdf 文件的内容加载到参数服务器 -->  
  <param name="robot_description" textfile="$(find  
urdf_gazebo_test)/urdf/sample.urdf" />
```

```
<!-- 启动 gazebo -->
<include file="$(find gazebo_ros)/launch/empty_world.launch" />
<!-- 在 gazebo 中显示机器人模型 -->
<node pkg="gazebo_ros" type="spawn_model" name="model"
args="-urdf -model mycar -param robot_description" />
</launch>
```

6.2 URDF 集成 Gazebo 的相关设置

相比于 Rviz 集成 URDF, Gazebo 在集成 URDF 时, 需要做一些修改, 比如: 必须添加 collision 碰撞属性相关参数、必须添加 inertial 惯性矩阵相关参数, 另外, 如果直接移植 Rviz 中机器人的颜色设置是没有显示的, 颜色设置也必须做相应的变更。

(1)、collision

如果机器人 link 是标准的几何体形状, 和 link 的 visual 属性设置一致即可。

(2)、inertial

惯性矩阵的设置需要结合 link 的质量与外形参数动态生成, 标准的球体、圆柱与立方体的惯性矩阵公式如下(已经封装为 xacro 实现):

```
<!-- Macro for inertia matrix -->
<xacro:macro name="sphere_inertial_matrix" params="m r">
  <inertial>
    <mass value="{m}" />
    <inertia ixx="{2*m*r*r/5}" ixy="0" ixz="0"
      iyy="{2*m*r*r/5}" iyz="0"
      izz="{2*m*r*r/5}" />
  </inertial>
</xacro:macro>
```

球体惯性矩阵

```
<xacro:macro name="cylinder_inertial_matrix" params="m r h">
  <inertial>
    <mass value="{m}" />
    <inertia ixx="{m*(3*r*r+h*h)/12}" ixy="0" ixz="0"
      iyy="{m*(3*r*r+h*h)/12}" iyz="0"
      izz="{m*r*r/2}" />
  </inertial>
</xacro:macro>
```

圆柱惯性矩阵


```
<xacro:macro name="Box_inertial_matrix" params="m l w h">
  <inertial>
    <mass value="{m}" />
    <inertia ixx="{m*(h*h + l*l)/12}" ixy = "0" ixz = "0"
      iyy="{m*(w*w + l*l)/12}" iyz = "0"
      izz="{m*(w*w + h*h)/12}" />
  </inertial>
</xacro:macro>
```

立方体惯性矩阵

需要注意的是，原则上，除了 base_footprint 外，机器人的每个刚体部分都需要设置惯性矩阵，且惯性矩阵必须经计算得出，如果随意定义刚体部分的惯性矩阵，那么可能会导致机器人在 Gazebo 中出现抖动，移动等现象。

(3)、颜色设置

在 Gazebo 中显示 link 的颜色，必须要使用指定的标签：

```
<gazebo reference="link 节点名称">
  <material>Gazebo/Blue</material>
</gazebo>
```

material 标签中，设置的值区分大小写，颜色可以设置为 Red Blue Green Black ...

6.3 Gazebo 仿真环境搭建

默认情况下，在 Gazebo 中机器人模型是在 empty world 中，并没有类似于房间、家具、道路、树木... 之类的仿真物，如何在 Gazebo 中创建仿真环境呢？

Gazebo 中创建仿真实现方式有两种：

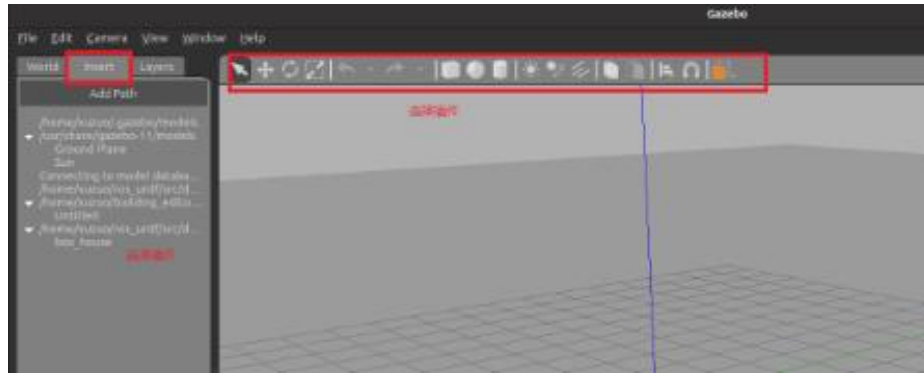
方式 1：直接添加内置组件创建仿真环境

方式 2：手动绘制仿真环境(更为灵活)

另外还有方式 3：可以直接下载使用官方或第三方提高的仿真环境插件。

方式 1、添加内置组件创建仿真环境

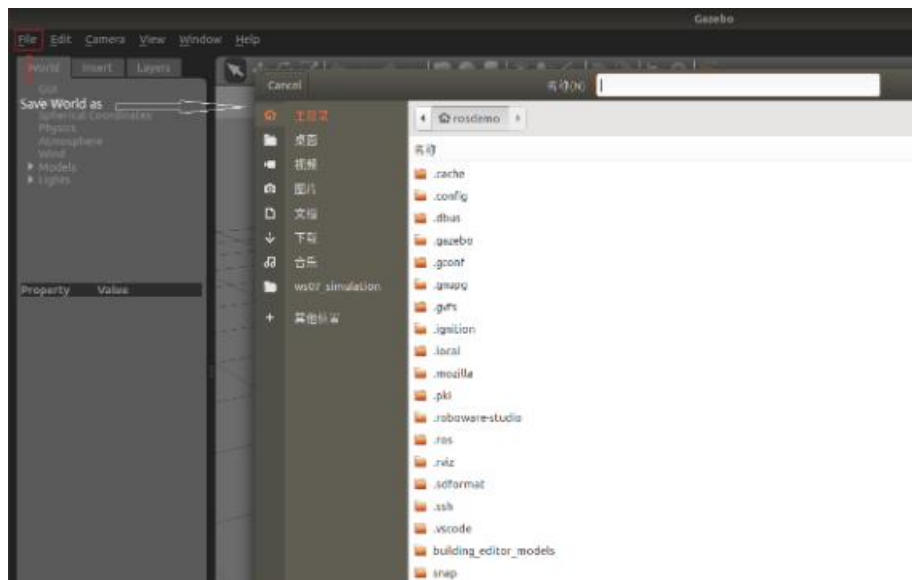
(1) 启动 Gazebo 并添加组件



(2) 保存仿真环境

添加完毕后，选择 file ---> Save World as 选择保存路径

(功能包下: worlds 目录)，文件名自定义，后缀名设置为.world



(3) 启动 launch 文件

```
<launch>
  <!-- 将 Urdf 文件的内容加载到参数服务器 -->
  <param name="robot_description" command="$(find xacro)/xacro $(find
urdf_gazebo)/urdf/xacro/my_base_camera_laser.urdf.xacro" />
  <!-- 启动 gazebo -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="world_name" value="$(find urdf_gazebo)/worlds/hello.world" />
  </include>

  <!-- 在 gazebo 中显示机器人模型 -->
  <node pkg="gazebo_ros" type="spawn_model" name="model" args="-urdf -model mycar -param robot_description" />
</launch>
```

核心代码: 启动 empty_world 后，再根据 arg 加载自定义的仿真环境。

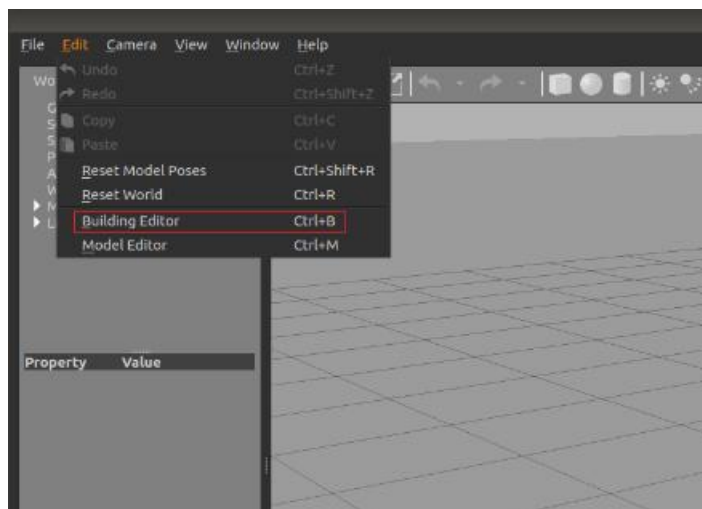
方式 2、自定义仿真环境

(1) 启动 Gazebo 打开构建面板，绘制仿真环境

(2) 保存构建的环境

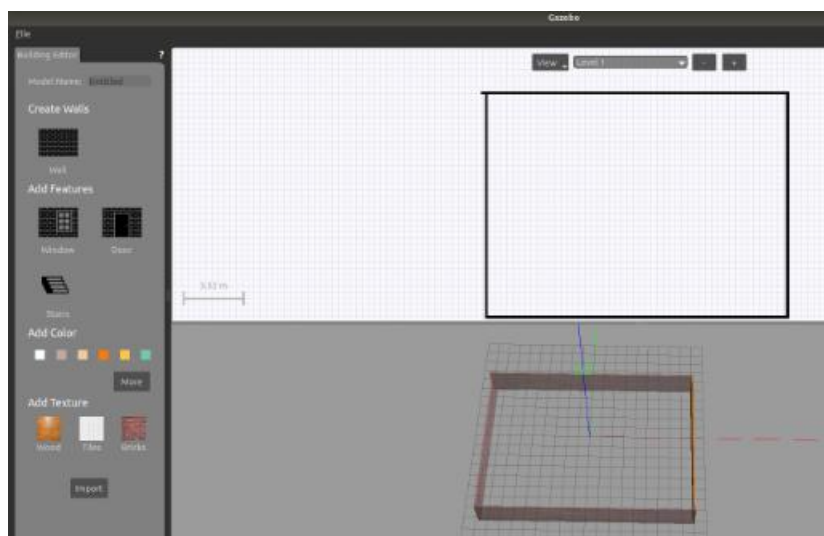
点击左上角 file 菜单的子菜单：Save (保存路径为功能包下的 models 目录)

然后点击 file 菜单的子菜单：Exit Building Editor



(3) 保存为 world 文件

可以像方式 1 一样再添加一些插件,然后保存为 world 文件(保存路径为功能包下的 worlds 目录)



(4) 启动 launch 文件

启动方式和方式 1 中的相同。

6.4 使用官方提供的插件

当前 Gazebo 提供的默认仿真道具有限，可以下载官方支持，提供更为丰富的仿真实现，具体实现如下：

(1) 下载官方模型库

```
git clone https://github.com/osrf/gazebo_models
```

注意：此过程可能比较耗时。

(2) 将模型库复制进 gazebo

将得到的 gazebo_models 文件夹内容复制到 /usr/share/gazebo-*/models 目录下。

(3) 应用

重启 Gazebo，选择左侧菜单栏的 insert，就可以选择并插入相关道具了。

7

URDF、Gazebo 和 Rviz 的综合应用

URDF 用于创建机器人模型，Rviz 可以显示机器人感知到的环境信息，Gazebo 用于仿真、可以模拟外界环境以及机器人的一些传感器。接下来介绍在实际的场景中如何将三者结合：

通过 Gazebo 模拟机器人的传感器，然后在 Rviz 中显示这些传感器感知到的数据。

主要的应用场景包括：

(1) 运动控制以及里程计信息显示

(2) 雷达信息仿真以及显示

(3) 摄像头信息仿真以及显示

(4) 深度相机的信息仿真以及显示

7.1 运动控制以及里程计信息显示应用

(1)、ros_control

ros_control 是一组软件包，它包含了控制器接口，控制器管理器，传输和硬件接口。

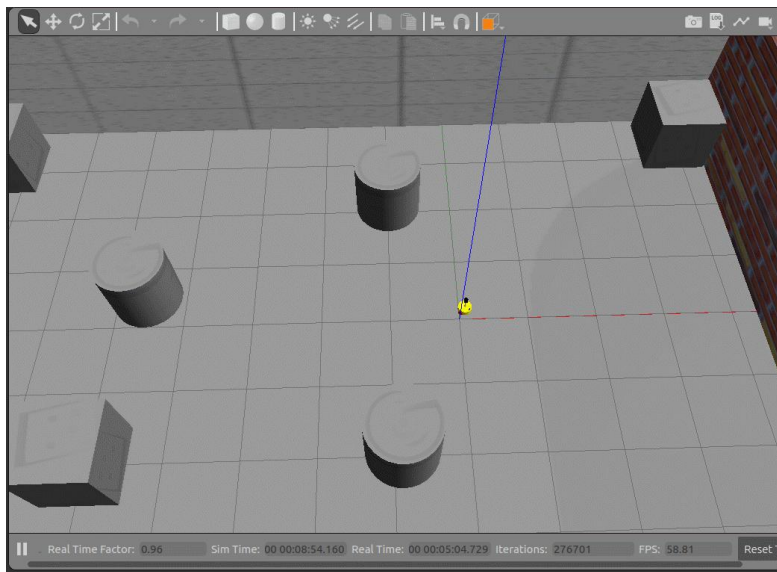
Gazebo 已经实现了 ros_control 的相关接口，如果需要在 Gazebo 中控制机器人运动，直接调用相关接口即可。

(2)、运动控制基本流程:

已经创建完毕的机器人模型，编写一个单独的 xacro 文件，为机器人模型添加传动装置以及控制器。

将此文件集成进 xacro 文件。

启动 Gazebo 并发布“/cmd_vel”话题消息来控制机器人运动。



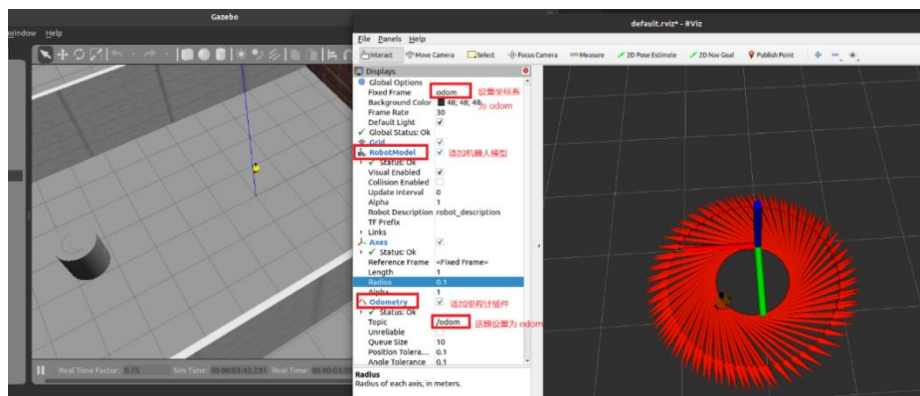
(3)、Rviz 查看里程计信息

在 Gazebo 的仿真环境中，机器人的里程计信息以及运动朝向等信息是无法获取的，可以通过 Rviz 显示机器人的里程计信息以及运动朝向。

里程计记录的是机器人相对出发点坐标系的位姿状态(X 坐标，Y 坐标，Z 坐标以及朝向)。

首先，启动 Rviz。

其次，在 Rviz 种添加图示组件。



7.2 雷达信息仿真以及显示

(1)、已经创建完毕的机器人模型，新建 Xacro 文件，配置雷达传感器信息，参见下图。

```
<?xml version="1.0" encoding="UTF-8" ?>
<robot name="my_sensors" xmlns:xacro="http://wiki.ros.org/xacro">
  <!-- 雷达 -->
  <gazebo reference="laser">
    <sensor type="ray" name="rplidar">
      <pose>0 0 0 0 0 0</pose>
      <visualize>true</visualize>
      <update_rate>5.5</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1</resolution>
            <min_angle>-3</min_angle>
            <max_angle>3</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.10</min>
          <max>30.0</max>
          <resolution>0.01</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>
      <plugin name="gazebo_rplidar" filename="libgazebo_ros_laser.so">
        <topicName>/scan</topicName>
        <frameName>laser</frameName>
      </plugin>
    </sensor>
  </gazebo>
</robot>
```

(2)、Xacro 文件集成

将 (1) 中的 Xacro 文件集成进总的机器人模型文件，代码示例如下：

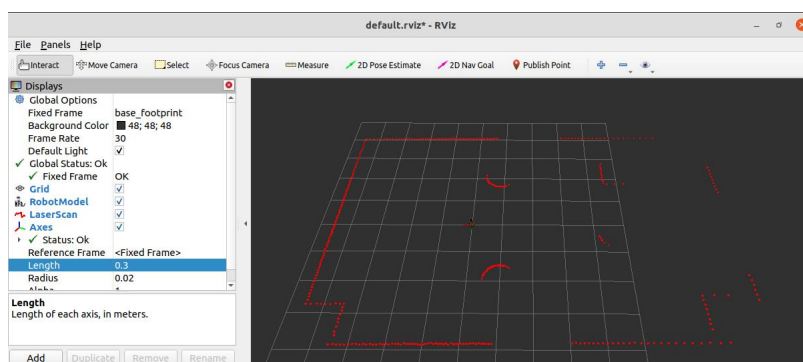
```
<!-- 组合小车底盘与传感器 -->
<robot name="my_car_camera" xmlns:xacro="http://wiki.ros.org/xacro">
  <xacro:include filename="my_head.urdf.xacro" />
  <xacro:include filename="my_base.urdf.xacro" />
  <xacro:include filename="my_camera.urdf.xacro" />
  <xacro:include filename="my_laser.urdf.xacro" />
  <xacro:include filename="move.urdf.xacro" />
  <!-- 雷达仿真的 xacro 文件 -->
  <xacro:include filename="my_sensors_laser.urdf.xacro" />
</robot>
```

(3)、启动仿真环境

编写 launch 文件，启动 Gazebo。

(4)、在 Rviz 中显示雷达数据

先启动 Rviz，添加雷达信息显示插件，配置好之后，可以看到红点点，就是雷达的扫描数据。



7.3 摄像头信息仿真以及显示

通过 Gazebo 模拟摄像头传感器，并在 Rviz 中显示摄像头数据。

(1)、在已经创建完毕的机器人模型基础上，新建 Xacro 文件，为模型添加摄像头配置，如下图所示。

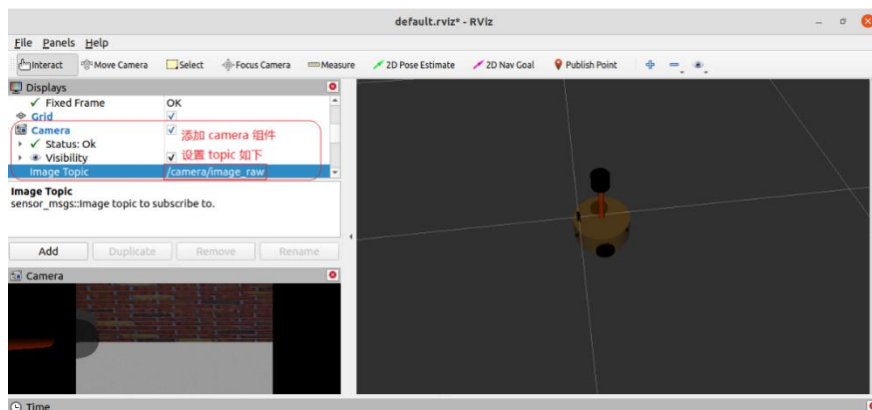

```
<robot name="my_sensors" xmlns:xacro="http://wiki.ros.org/xacro">
  <!-- 被引用的link -->
  <gazebo reference="camera">
    <!-- 类型设置为 camera -->
    <sensor type="camera" name="camera_node">
      <update_rate>30.0</update_rate> <!-- 更新频率 -->
      <!-- 摄像头基本信息设置 -->
      <camera name="head">
        <horizontal_fov>1.3962634</horizontal_fov>
        <image>
          <width>1280</width>
          <height>720</height>
          <format>R8G8B8</format>
        </image>
        <clip>
          <near>0.02</near>
          <far>300</far>
        </clip>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.007</stddev>
        </noise>
      </camera>
    </sensor>
    <!-- 核心插件 -->
    <plugin name="gazebo_camera" filename="libgazebo_ros_camera.so">
      <alwaysOn>true</alwaysOn>
      <updateRate>0.0</updateRate>
      <cameraName>/camera</cameraName>
      <imageTopicName>image_raw</imageTopicName>
      <cameraInfoTopicName>camera_info</cameraInfoTopicName>
      <frameName>camera</frameName>
      <hackBaseline>0.07</hackBaseline>
      <distortionK1>0.0</distortionK1>
      <distortionK2>0.0</distortionK2>
      <distortionK3>0.0</distortionK3>
      <distortionT1>0.0</distortionT1>
      <distortionT2>0.0</distortionT2>
    </plugin>
  </gazebo>
</robot>
```

(2)、将步骤 1 的 Xacro 文件集成进总的机器人模型文件，代码示例如下图。

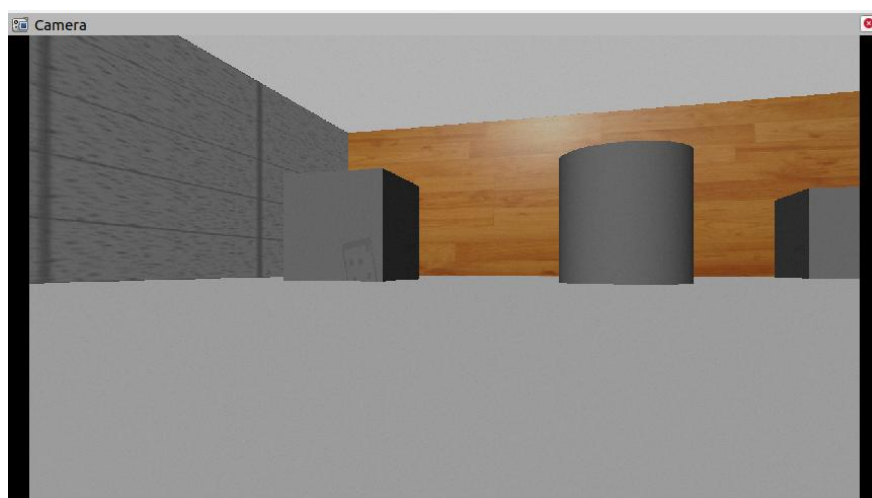
```
<!-- 组合小车底盘与传感器 -->
<robot name="my_car_camera" xmlns:xacro="http://wiki.ros.org/xacro">
  <xacro:include filename="my_head.urdf.xacro" />
  <xacro:include filename="my_base.urdf.xacro" />
  <xacro:include filename="my_camera.urdf.xacro" />
  <xacro:include filename="my_laser.urdf.xacro" />
  <xacro:include filename="move.urdf.xacro" />
  <!-- 摄像头仿真的 xacro 文件 -->
  <xacro:include filename="my_sensors_camara.urdf.xacro" />
</robot>
```

(3)、编写 launch 文件，启动 Gazebo。

(4)、启动 Rviz，在 Rviz 中添加摄像头组件。



摄像头的的数据展示效果如下：



7.4 深度相机信息仿真以及显示

通过 Gazebo 模拟 kinect 深度摄像头，并在 Rviz 中显示 kinect 深度摄像头数据。

(1)、在已经创建好的机器人模型基础上，新建 Xacro 文件为模型添加 kinect 深度摄像头配置；

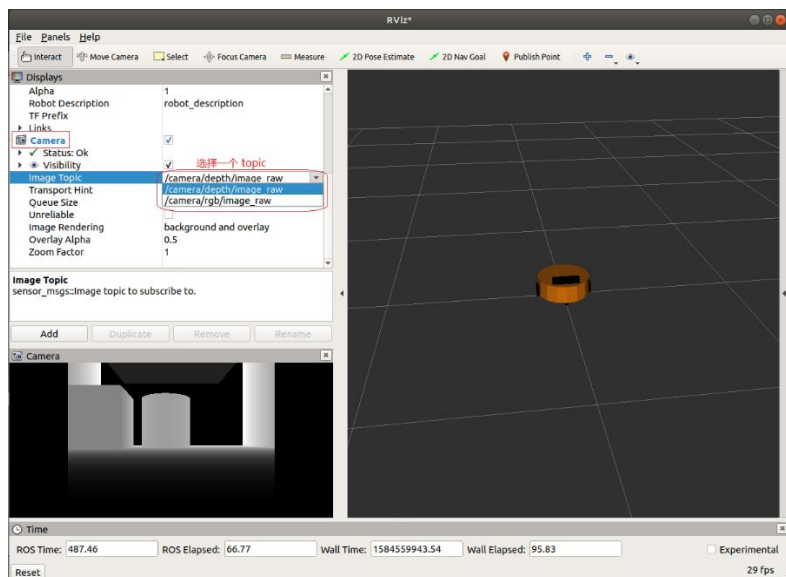
```
<robot name="my_sensors" xmlns:xacro="http://wiki.ros.org/xacro">
  <gazebo reference="kinect link名称">
    <sensor type="depth" name="camera">
      <always_on>true</always_on>
      <update_rate>20.0</update_rate>
      <camera>
        <horizontal_fov>${60.0*PI/180.0}</horizontal_fov>
        <image>
          <format>RGB888</format>
          <width>640</width>
          <height>480</height>
        </image>
        <clip>
          <near>0.05</near>
          <far>8.0</far>
        </clip>
      </camera>
      <plugin name="kinect_camera_controller" filename="libgazebo_ros_openni_kinect.so">
        <cameraName>camera</cameraName>
        <alwaysOn>true</alwaysOn>
        <updateRate>10</updateRate>
        <imageTopicName>rgb/image_raw</imageTopicName>
        <depthImageTopicName>depth/image_raw</depthImageTopicName>
        <pointCloudTopicName>depth/points</pointCloudTopicName>
        <cameraInfoTopicName>rgb/camera_info</cameraInfoTopicName>
        <depthImageCameraInfoTopicName>depth/camera_info</depthImageCameraInfoTopicName>
        <frameName>kinect link名称</frameName>
        <baseline>0.1</baseline>
        <distortion_k1>0.0</distortion_k1>
        <distortion_k2>0.0</distortion_k2>
        <distortion_k3>0.0</distortion_k3>
        <distortion_t1>0.0</distortion_t1>
        <distortion_t2>0.0</distortion_t2>
        <pointCloudCutoff>0.4</pointCloudCutoff>
      </plugin>
    </sensor>
  </gazebo>
</robot>
```

(2)、将步骤 1 的 Xacro 文件集成进总的机器人模型文件，代码示例如下：

```
<!-- 组合小车底盘与传感器 -->
<robot name="my_car_camera" xmlns:xacro="http://wiki.ros.org/xacro">
  <xacro:include filename="my_head.urdf.xacro" />
  <xacro:include filename="my_base.urdf.xacro" />
  <xacro:include filename="my_camera.urdf.xacro" />
  <xacro:include filename="my_laser.urdf.xacro" />
  <xacro:include filename="move.urdf.xacro" />
  <!-- kinect仿真的 xacro 文件 -->
  <xacro:include filename="my_sensors_kinect.urdf.xacro" />
</robot>
```

(3)、编写 launch 文件，启动 Gazebo

(4)、启动 Rviz，添加摄像头组件查看 kinect 深度相机数据。



kinect 深度摄像头的数据展示效果如下：

